ÖZYEĞİN UNİVERSİTY
FACULTY OF ENGINEERING

CS400

SUMMER PRACTICE REPORT

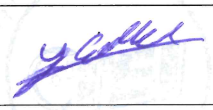
OĞUZ MERT COŞKUN
S034085

INTERNSHIP COMPANY & DEPARTMENT:

TEİ TUSAŞ MOTOR SANAYİİ A.Ş.
&
SOFTWARE DEPARTMENT

15.08.2025

SUMMER PRACTICE REPORT

STUDENT	
Name	Oğuz Mert Coşkun
Internship Start Date	21 /07 /2025
Internship Completion Date	15 /08 /2025
Total Working Days	20
COMPANY	
Name	TEİ TUSAŞ MOTOR SANAYİİ A.Ş.
Department	Software Development
Address	Esentepe Mahallesi Çevrelyolu Blv. No:356 Tepebaşı/Eskişehir
SUPERVISOR	
Name	Yağız ÇAKMAK
Title	Embedded Software Engineer
Department	Software Department
Phone	05365119999
E-Mail	yagiz.cakmak@tei.com.tr
Signature	

DAILY WORK SUMMARY

DAY	DATE	WORK DESCRIPTION
1	21.07.2025	Attended orientation presentation.
2	22.07.2025	Had an Occupational Health and Safety (OHS) education.
3	23.07.2025	Had an Occupational Health and Safety (OHS) education. The work environment was inspected.
4	24.07.2025	Met my mentor and new colleagues. Had a tour of the buildings and learned which buildings have which units are in there.
5	25.07.2025	Discussed and finalized a project topic with my mentor. Began preliminary research for the project.
6	28.07.2025	Studied Aviation Standards, scheduling algorithms, and SecureBoot.
7	29.07.2025	Studied Memory Layout, Bootloaders to understand flash memory. Also searched what happens when the reset button is pressed.
8	30.07.2025	Studied the I2C protocol to learn how to establish I2C communication between an IMU sensor and an STM32 board.
9	31.07.2025	Studied Flash memory and Linker File. Programmed a mini project to learn how to implement Flash Memory.
10	01.08.2025	Programmed a mini project to learn how to implement Flash Memory.

Student's Name: Oğuz Mert Coşkun

Supervisor's Name: Yağız Çakmak

Student's Signature:

Supervisor's Signature-Stamp:

DAILY WORK SUMMARY

DAY	DATE	WORK DESCRIPTION
11	04.08.2025	Studied MISRA C(2012 Edition) and read Software Standards to learn how to write clean code. Programmed a mini project to learn the I2C protocol by receiving sensor readings from the IMU sensor.
12	05.08.2025	Completed reading Software Standard. Could compute acceleration and gyro readings into pitch and roll from an IMU sensor.
13	06.08.2025	Studied Basic Timers to receive sensor readings at a desired period. Created new project to implement timers for the project.
14	07.08.2025	Scheduling methods were inspected. Fixed the Timer and I2C errors.
15	08.08.2025	Added RTC to store the IMU readings and current time into a struct. Implemented 8 8-byte-sized structs for logging Flash memory and integrated Flash methods.
16	11.08.2025	Implemented SysTick to measure the execution time of code's parts. After the button is pressed, the next 150 samples are received with 10-ms periods. Then the samples were logged into flash memory one by one every 10ms.
17	12.08.2025	An additional 9 Flash pages were initialized to store the entire 250 samples in a flash page per button press without occupying full pages. Added a circular buffer to update the oldest data with incoming data every 10 ms.
18	13.08.2025	Whole 250 samples logging into Flash Memory is complete. The first 100 samples were copied from the circular buffer to write 100 samples in a single time. The next 150 samples were written into Flash memory one by one every 10 ms.
19	14.08.2025	Implemented Independent Watchdog(IWDG) to learn the concept of Watchdog Timers. Prepared for the project presentation.
20	15.08.2025	Did a presentation about my project.

Student's Name: Oğuz Mert Coşkun

Supervisor's Name: Yağız Çakmak

Student's Signature: 

Supervisor's Signature: 

Abstract

I completed a 20-day internship at TEİ TUSAŞ Motor Sanayii A.Ş., in the Software Development Department at the Eskişehir Headquarters. My project involved developing an incident recording system using an STM32 microcontroller and an MPU6050 IMU sensor. The system was designed to capture IMU data sampled every 10 ms, storing 100 samples before and 150 samples after a button press into Flash memory.

To achieve this, I implemented scheduling with a circular buffer for efficient data handling and used interrupts for precise timing and event control. The project adhered to DO-178C Aviation and MISRA-C standards, ensuring safety and code quality. Development was carried out in STM32CubeIDE and STM32CubeMX, with validation through Flash memory tracking.

This project provided practical experience in embedded software development, including incident recording, Flash management, and hardware/software interfacing, while enhancing my skills in C programming, real-time systems, STM32 peripherals, and safety-critical coding practices.

I. Introduction

I completed a 20-day internship at TEİ TUSAŞ Motor Sanayii A.Ş. My main task was to store IMU readings sampled every 10 ms, starting from 1 second before and continuing until 1.5 seconds after a user button press. Once all the samples were collected, they were saved into designated Flash Bank pages in the microcontroller's Flash memory. The project was assigned to me because of my interest in embedded software engineering and learning more about Incident Recording, Scheduling, Memory Maps and Timers. During this project, I used DO-178C Aviation and MISRA-C (2012 Edition) Standards to learn how to follow safety procedures and to write clean code. I have used STM32 microcontrollers and an IMU sensor for the project.

II. Company Description

TUSAŞ Engine Industries, Inc. (TEI) was established in 1985 as a joint venture between Turkish Aerospace Industries (TAI), General Electric (GE), the Turkish Armed Forces Foundation (TSKGV), and the Turkish Aeronautical Association (THK). By delivering high-quality products and services to the aviation industry, TEI has become an international manufacturer and a global design center, steadily advancing toward its goals. In 1987, TEI carried out its first production and delivery of engines and engine parts. Over the years, it has successfully transferred aircraft engine component and module production, assembly, and testing technologies to Türkiye, while proving its reliability and high-quality manufacturing capabilities in global markets in line with international standards*(TEI, n.d.)*.

TEI (TUSAŞ Engine Industries, Inc.) designs, develops, and manufactures advanced engines and critical components for both civil and military aviation. Its product range includes indigenous engines such as the TEI-TS1400 turboshaft powering the T625 Gökbey helicopter, the TEI-TF6000 turbofan developed for future unmanned aerial systems, and the TEI-PD170 turbodiesel engine used in UAVs. In addition to complete engine programs, TEI also produces and maintains parts for global aviation leaders, supporting commercial aircraft engines like GE's LEAP and GEnx. Through these projects, TEI contributes to Türkiye's aviation self-sufficiency while maintaining a strong position in international supply chains *(TEI, n.d.)*.

During my internship, I was assigned to a department under TEI Eskisehir Headquarters. The department I was assigned was Software Development which focused on Embedded Software Engineering. In the Software Development department, which consists of approximately 30 engineers, they write code for drivers, embedded software verification and create tools for computer programs. My role is to make a project that consists of incident recording, scheduling and timers. That's why, I was assigned to store IMU sensor readings into Flash Memory when user button is pressed.

III. Incident Record using Scheduling

i. Problem Statement

My mentor wanted me to learn how the programs are initialized, and which files are running after a reset button is pressed or power is cut. Therefore, he wanted me to use Flash Memory for storing incoming readings from an IMU sensor. The sensor readings from the GY521-MPU6050 are received in every 10 ms. The sensor readings are stored in Flash memory's pages according to press time of user buttons starting from 1 second before and continuing until 1.5 seconds after a user button press. But storing every IMU reading is not time efficient. That's why we added an User Button to set a timepoint. The IMU readings with RTC values starting from 1 second before and continuing until 1.5 seconds after a user button press is going to be stored in Flash Memory*(See Figure 1)*.

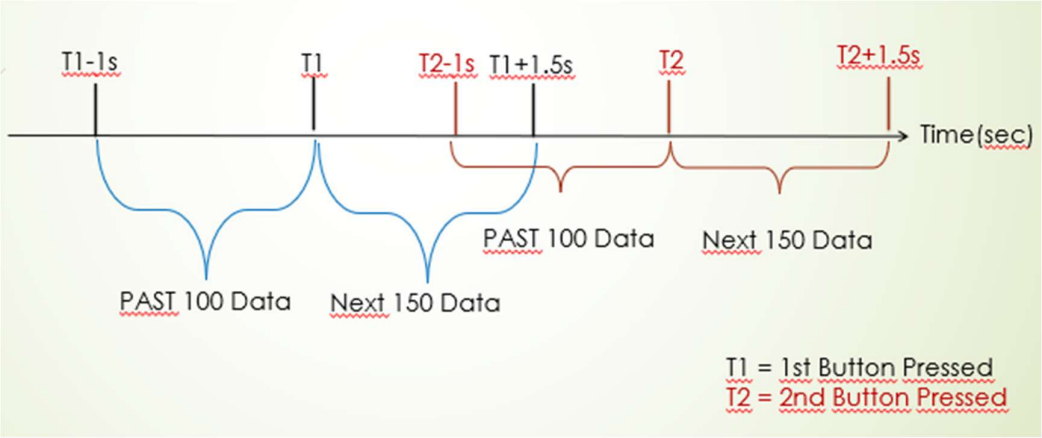


Figure 1 Timeline of per button

ii. Tools and Techniques Used

ii.1. Programming Languages:

I programmed embedded systems using C and C++ during my internship, specifically the STM32. These languages have an advantage over other languages because of their low-level control over hardware, which makes them extremely efficient for embedded systems. STM32 was selected for the project because it is better suited for complicated embedded applications than Arduino and provides advanced functionality at a lower cost than alternatives like the Raspberry Pi. In addition, having experience in STM32, was a reason to go with STM32 since I had 3 weeks to complete the project.

To program an STM32 development kit, STM32CubeIDE and STM32CubeMX are required*(See Figure 2 and 3)*. According to STMicroelectronics, STM32CubeIDE is a

complete C/C++ development environment for STM32 microcontrollers and microprocessors. . It is an advanced tool for embedded system development since it offers debugging features, code generation, code compilation, and peripheral configuration. In order to facilitate effective development and troubleshooting, the platform also offers a variety of conventional and advanced debugging features that enable users to view memory, peripheral registers, and CPU core registers *(STMicroelectronics, n.d., para. 1).*

STM32CubeMX is a graphical tool that simplifies the configuration of STM32 microcontrollers and microprocessors. The tool enables users to define GPIOs, set up the system clock, and interactively assign peripherals such as I2C and timers, making complicated systems efficient.

ii.2. Software:

The two primary libraries for programming STM32 are Low-Layer (LL) and HAL. Because according to the website the LL library is lightweight and directly accesses hardware registers, it takes up less memory and provides faster performance as well as more precise hardware setting control with less overhead and delay *(Introduction to hal library and LL Library,nd)*. On the other hand, by abstracting low-level register operations, the HAL library provides a more intuitive API and a higher level of abstraction, making it simpler to grasp *(Hal Vs ll, 2021)*.I chose to use HAL library instead of LL library even though I like to use LL library since my focus is to learn incident recording and scheduling.

ii.3. Hardware:

I used the STM32-G071RB development kit since I already had bought it*(See Figure 4)*. It meets the needs of Timers and I2C peripherals. For I2C protocol, I used GY521-MPU6050 IMU sensor since it measures acceleration and gyro in two dimensions which are X and Y axis with RTC values *(See Figure 5).* In addition, it is not costly option for a I2C sensor.

ii.4. Techniques:

To buffer incoming sensor readings, I used one of the non-preemptive scheduling methods that is FIFO. For implementation of the FIFO, I used circular buffer for storing incoming sensor readings. The circular buffer holds the newest 100 readings. Thanks to circular buffer, there is no data overflow or data loss.

Interrupt protocol is used for both User Button and Timer. When user button is pressed, the interrupt raises flag for starting the process. Timer raises flag for new data reception in 10 ms intervals.

Figure 2 STM32CubeIDE

Figure 3 STM32MX

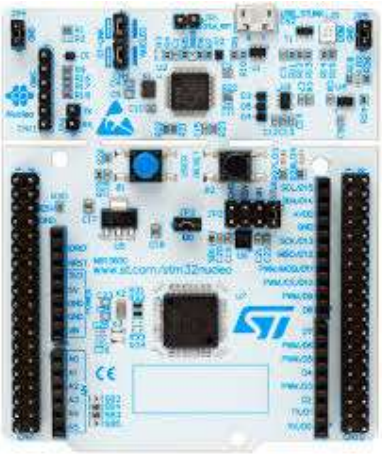


Figure 4 STM32-G071RB

Figure 5 GY521- MPU6050

iii. Detailed Explanation

Firstly, the hardware is set up by connecting GY-521 to Nucleo-G071RB. The GY-521 is powered must be powered with between 2.375 V and 3.46 V. That's why, the GY-521 is powered with 3.3V. The IMU sensor can have two I^2C addresses by controlling ADO pin;

- AD0 = LOW, I^2C address = 1101000,
- AD0 = HIGH, I^2C address = 1101001.

In order to set the I^2C address as 1101000, I connected AD0 pin to GND. According to datasheet, the D14 and D15 pins are connected to SDA, SCL pins of GY-521*(See Table 1)*.

PIN Connection	
Nucleo-G071RB	GY521-MPU6050
3V3	VCC
GND	GND
GND	AD0
D14	SDA
D15	SCL

TABLE 1: Hardware Pin Connection

After setting up the hardware connection. The STM32 pins are configured in STM32CubeMX to enable I2C peripheral, Timers and Interrupt*(See Figure 6 and 7)*. The Tim3 is used for set up period of 10 ms. In every 10 ms, Tim3 is interrupted to raise flag for receiving new sensor reading.

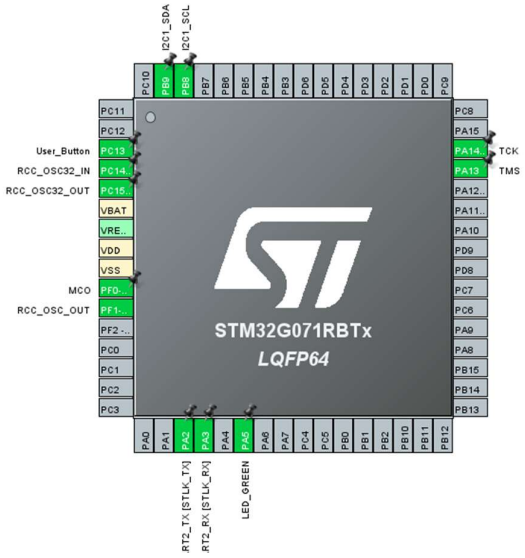


Figure 6 The pinout view of STM32GO71RBTx

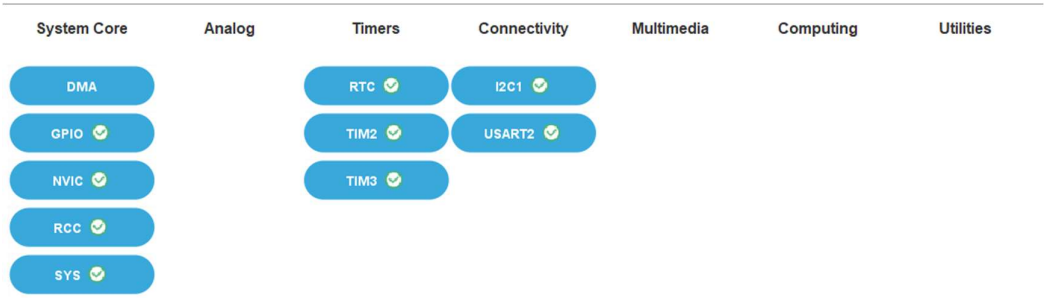


Figure 7 System view of Nucleo-G071RB

The TIM3 is set to raise a tick every 10 ms. The necessary calculations are made for ARR and PSC values.

$$Timeout = \frac{(ARR+1)(PSC+1)}{fCLK}, if\ ARR = 39999\ and\ PSC = 15$$

$$Tout = \frac{(39999+1)(15+1)}{64000} = 10\ ms$$

Firstly, the GY-521 initialization method is programmed. According to data sheet, 0x6B register is set to 0x00 for waking the sensor up. The 0x19, 0x1B and 0x1C registers are for configuring the margin of error

```
void MPU6050_Init(void)
{
	uint8_t data =0x00;

	//Wake sensor up -> register 0x6B
	HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR <<1, 0x6B, 1, &data, 1, 1000);

	//SET DATA RATE of 1Khz -> SMPLRT_DIV register
	data = 0x07;
	HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR <<1, 0x19, 1, &data, 1, 1000);

	//GYRO_CONFIG <strong>±</strong> 250 °/s
	data = 0x00;
	HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR <<1, 0x1B, 1, &data, 1, 1000);
```

```
        //ACCEL_CONFIG  <strong>±</strong> 2g
        data = 0x00;
        HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR <<1, 0x1C, 1, &data, 1, 1000);
}
```

The peripherals(I2C, Tim3 and RTC) are initialized. After initialization of peripherals, the first loop checks the I2C address of Gy-521. If the I2C address is found, it means that the sensor is connected to the MCU. Since MPU6050 is connected to MCU, the initialization of MPU6050 is done. Now MPU6050 is ready for receiving input and transmitting incoming sensor readings through I2C protocol.

```
/* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_USART2_UART_Init();
  MX_I2C1_Init();
  MX_TIM2_Init();
  MX_RTC_Init();
  MX_TIM3_Init();
  /* USER CODE BEGIN 2 */
  for(int addr = 0 ; addr<128 ; addr++){
        HAL_StatusTypeDef ret = HAL_I2C_IsDeviceReady(&hi2c1, addr <<1,
                                                      1, 1000);
        if(ret == HAL_OK)
        {
              printf("MPU is ready at I2C address: 0x%02X\n", addr);
         }
        else
        {
              printf("MPU is not ready. Check it.\n");
        }
  }
  HAL_TIM_Base_Start_IT(&htim3);

  //Initalize MPU6050
  MPU6050_Init();
```

GY-521 does only read Acceleration and Gyro in three dimensions(X,Y and Z axes). That's why, there is two method "MPU6050_Read_Accel" and "MPU6050_Read_Gyro" which reads acceleration and gyro. In order to read acceleration and gyro, the certain 2-byte sized registers that stores readings must be read and merged into a variable*(See Appendix B)*. Since MPU6050 doesn't consist of magnetometer, we can only calculate the pitch and roll in X and Y axes.

```
void MPU6050_Read_Accel(void)
{  //Execution Time = 1.2ms
       uint8_t Accel_Data[6];

       // Read 6 BYTES of data starting from ACCEL_XOUT_H (0x3B) register
       HAL_I2C_Mem_Read (&hi2c1, MPU6050_ADDR <<1 , 0x3B, 1, Accel_Data,
                                                      6, 1000);
       g_intAccelXRaw = (int16_t)(Accel_Data[0] << 8 | Accel_Data[1]);
       g_intAccelYRaw = (int16_t)(Accel_Data[2] << 8 | Accel_Data[3]);
       g_intAccelZRaw = (int16_t)(Accel_Data[4] << 8 | Accel_Data[5]);

       // Compute pitch and roll
       g_fltRollAcceleration  = atan2((float)g_intAccelYRaw,
                                      (float)g_intAccelZRaw) * 180.0 / M_PI;
```

```
	g_fltPitchAcceleration = atan2(-(float)g_intAccelXRaw,
sqrt(g_intAccelYRaw * g_intAccelYRaw + g_intAccelZRaw * g_intAccelZRaw)) *
                                                              180.0 / M_PI;
}
```

Doxygen style-comments are written above of all the methods. The Doxygen comments explain what does the method accomplish, which buffers are used and whether any precondition to use the method or whether it returns any value.

```
/**
 * MPU6050_Read_Gyro
 *
 * This function reads raw gyroscope data from the MPU6050 and converts it
to angular velocity.
 *
 * This function communicates with the MPU6050 sensor via I2C to read
 * 6 bytes of raw gyroscope data (X, Y, Z axes) strating from GYRO_XOUT_h
register,
 * combines them into signed 16-bit integers, and converts them into angular
velocities
 * in degrees per second using the sensitivity scale factor for ±250°/s
range (131 LSB/°/s)
 *
 * Buffer : Gx, Gy, Gz
 *
 * @Precondition: N/A
 *
 * @Return: N/A
 */
void MPU6050_Read_Gyro (void)
{
	uint8_t Gyro_Data[6];
	// Read 6 BYTES of data starting from GYRO_XOUT_H register
	 HAL_I2C_Mem_Read (&hi2c1, MPU6050_ADDR <<1, 0x43, 1, Gyro_Data,
                                                          6, 1000);

	g_intGyroXRaw = (int16_t)(Gyro_Data[0] << 8 | Gyro_Data[1]);
	g_intGyroYRaw = (int16_t)(Gyro_Data[2] << 8 | Gyro_Data[3]);
	g_intGyroZRaw = (int16_t)(Gyro_Data[4] << 8 | Gyro_Data[5]);

	Gx = (float)g_intGyroXRaw/131.0;
	Gy = (float)g_intGyroYRaw/131.0;
	Gz = (float)g_intGyroZRaw/131.0;
}
```

The UserButton_Callback interrupts when User Button is pressed and unless the flash memory writing process is busy, it raises flag that tells the session is ready for processing.

```
void UserButton_Callback(void){
	if(!g_int_LogSessionBusyFlag){
		g_untIOButtonPressed=1;
	}
	//Toggle the GREEN_LED
	HAL_GPIO_TogglePin(GREEN_LED_GPIO_Port, GREEN_LED_Pin);
}
```

Since 250 data is going to be stored in a Flash Page that its capacity is 2KB, if I assumed all the data structure is 8 byte sized, then maximum 256 data can be stored per page(See Appendix B).

$$\text{Max Number of } 8-\text{byte sized structs per Page} = \frac{2\text{KB}}{8\text{ Bytes}} = \frac{2048\text{ Bytes}}{8\text{ Bytes}} = 256\text{ data}$$

That's why, Construct_Flash_Struct method, constructs the data structure to prepare for parsing into Flash Memory.

- int16_t intProcessedPitch – 2 bytes
- int16_t intProcessedRoll – 2 bytes
- uint8_t intRTCHours – 1 byte
- uint8_t intRTCMinutes – 1 byte
- uint8_t intRTCSeconds – 1 byte
- uint8_t intPadding – 1 byte
- Total size: 8 bytes

The last bytes is added as padding. Even though the software adds 1 byte padding to ensure layout, I defined padding to show the structure of the constructed data properly.

```
void Construct_Flash_Struct(tsCurrentFlashStruct *data)
{
        RTC_TimeTypeDef sTime;
        RTC_DateTypeDef sDate;

      HAL_RTC_GetTime(&hrtc, &sTime, RTC_FORMAT_BIN);
      HAL_RTC_GetDate(&hrtc, &sDate, RTC_FORMAT_BIN); // Unlocks the shadow
                                                         registers

      data->intProcessedPitch = (int16_t)(g_fltPitchAcceleration * 100);
      data->intProcessedRoll  = (int16_t)(g_fltRollAcceleration  * 100);
      data->intRTCHours   = sTime.Hours;
      data->intRTCMinutes = sTime.Minutes;
      data->intRTCSeconds = sTime.Seconds;
      data->intPadding = 0;
}
```

A circular buffer is used to store the newest 100 value. In every 10ms, the Dynamic_Buffer_Write is called. The method inserts the new value at the current head position and advances the g_untCircularBufferHead pointer in a circular manner. If the buffer is not yet full, the g_untCircularBufferSizeFilled is incremented. If the buffer is full, the oldest value (at the tail) is overwritten and the g_untCircularBufferTail pointer is advanced.

```
void Dynamic_Buffer_Write(uint64_t value)
{
    g_arr_untDynamicBuffer[g_untCircularBufferHead] = value;
    g_untCircularBufferHead = (g_untCircularBufferHead + 1) %
                                                    DYNAMIC_BUFFER_SIZE;
    if (g_untCircularBufferSizeFilled < DYNAMIC_BUFFER_SIZE)
    {
        g_untCircularBufferSizeFilled++;
    }
    else
    {
        // buffer is full, overwrite oldest
        g_untCircularBufferTail = (g_untCircularBufferTail + 1) %
                                                    DYNAMIC_BUFFER_SIZE;
```

```
    }
}
```

When the data in circular buffer is required for parsing into Flash Memory, Dynamic_Buffer_Read is called . The method retrieves the value at the tail position, advances the g_untCircularBufferTail pointer, and decrements the g_untCircularBufferSizeFilled. If the buffer is empty, the function returns 0 and does not modify the output value.

```
uint64_t Dynamic_Buffer_Read(uint64_t *value)
{
    if (g_untCircularBufferSizeFilled == 0) return 0; // empty

    *value = g_arr_untDynamicBuffer[g_untCircularBufferTail];
    g_untCircularBufferTail = (g_untCircularBufferTail + 1) %
                                                    DYNAMIC_BUFFER_SIZE;
    g_untCircularBufferSizeFilled--;
    return 1;
}
```

After User Button is pressed, we already have the newest 100 data for parsing into Flash Memory. Firslty,the Page Start Address is generated according to number of button pressed. Then desired Flash Page is erased, since the filled parts can't be overwritten. Then whole page is written from the start with the contents stored in circular buffer.

```
void Flash_Write_Past_1s(void)
 {
        uint32_t    Page_Start_Addr=    (uint32_t)    FLASH_PAGE54_START    +
((g_int_ButtonPressedCount-1) * PAGE_SIZE);

        memcpy(g_arr_untFlashPageBuffer,     (uint32_t     *)Page_Start_Addr,
PAGE_SIZE);

        HAL_FLASH_Unlock();

        FLASH_EraseInitTypeDef erase_init =
        {
          .TypeErase    = FLASH_TYPEERASE_PAGES,
          .Banks        = 1,
          .Page         = 54 + (g_int_ButtonPressedCount-1),
                                                  // Last page for 128KB
          .NbPages      = 1
        };

          uint32_t page_error = 0;
          if (HAL_FLASHEx_Erase(&erase_init, &page_error) != HAL_OK)
          {
                Error_Handler();
          }

          uint64_t data;
          uint16_t indx=0;
          while(Dynamic_Buffer_Read(&data))
          {
                g_arr_untFlashPageBuffer[indx++] = data;
          }

   //Write the whole page
         g_untCurrentMemoryAddress = Page_Start_Addr;
```

```
        for (uint32_t i = 0 ; (i*8) < PAGE_SIZE; i++)
        {
                uint64_t data64 = g_arr_untFlashPageBuffer[i];
                if (HAL_FLASH_Program(FLASH_TYPEPROGRAM_DOUBLEWORD,
                      g_untCurrentMemoryAddress + (i*8), data64) != HAL_OK)
                {
                       // handle error
                       HAL_FLASH_Lock();
                       Error_Handler();
                       return;
                }
         }
        HAL_FLASH_Lock();

       g_untCurrentFlashAddress = Page_Start_Addr + (indx*8);
}
```

After writing the past 1 second data, the new data should be parsed into Flash page. The Flash_Write_After_2Secs parses new data into the flash page. Despite the fact that the flash page can't be overwritten, whole page needs to be erased. Thus, firstly the values stored in the page are stored in a g_arr_untFlashPageBuffer.Then Flash page is erased for writing. Firstly, the contents in g_arr_untFlashPageBuffer is written into their previous memory places. Finally, the new data is parsed.

```
void Flash_Write_After_2Secs(uint32_t current_pointer)
 {
       uint32_t    Page_Start_Addr=    (uint32_t)    FLASH_PAGE54_START    +
((g_int_ButtonPressedCount-1) * PAGE_SIZE);

       memcpy(g_arr_untFlashPageBuffer,     (uint32_t     *)Page_Start_Addr,
PAGE_SIZE);

       HAL_FLASH_Unlock();

         FLASH_EraseInitTypeDef erase_init =
         {
          .TypeErase   = FLASH_TYPEERASE_PAGES,
            .Banks       = 1,
            .Page        = 54 + (g_int_ButtonPressedCount-1),    //    Last
page for 128KB
          .NbPages     = 1
         };

         uint32_t page_error = 0;
         if (HAL_FLASHEx_Erase(&erase_init, &page_error) != HAL_OK)
         {
             Error_Handler();
         }

         //Get  the  index  for  the  next  data  to  be  updated  into  the
g_arr_untFlashPageBuffer
         uint16_t    index    =    (uint16_t)((g_untCurrentFlashAddress    -
Page_Start_Addr) / 8);

         g_arr_untFlashPageBuffer[index] = g_untDataToWrite;

         //Write the whole page
         g_untCurrentMemoryAddress = Page_Start_Addr;
         for (uint32_t i = 0 ; (i*8) < PAGE_SIZE; i++)
         {
```

```
                uint64_t data64 = g_arr_untFlashPageBuffer[i];
                if           (HAL_FLASH_Program(FLASH_TYPEPROGRAM_DOUBLEWORD,
g_untCurrentMemoryAddress + (i*8), data64) != HAL_OK) {
                      // handle error
                      HAL_FLASH_Lock();
                      Error_Handler();
                      return;
                }
          }
         HAL_FLASH_Lock();

         //Increment the pointer
         if(current_pointer >= Page_Start_Addr + PAGE_SIZE )
         {
               g_untCurrentFlashAddress = Page_Start_Addr;
         }
         else
         {
               g_untCurrentFlashAddress+=8;
         }
 }
```

When TIM3 generates an update event, the function sets the g_untTim3Tick to indicate that a timing interval has elapsed. If the g_int_SampleCounter is less than 150, the g_int_ReceiveIncomingDataFlag is set to enable data reception. Otherwise, data reception is stopped, the logging session is ended, and the system state is set to EN_STATE_IDLE.

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{

       if (htim->Instance == TIM3)
        {
              g_untTim3Tick=1;
              if(g_int_SampleCounter<150)
              {
                    g_int_ReceiveIncomingDataFlag = 1;
              }
              else
              {
                    g_int_ReceiveIncomingDataFlag=0;
                    g_int_LogSessionBusyFlag = 0;
                    g_untState=EN_STATE_IDLE;
              }

        }
 }
```

The while loop is controlled with user button, Tim3 and states of the process. According to states or interrupt flags the process is run.

```
  while (1)
  {
        if(g_untTim3Tick)
              {
                  // Prepare to write double word (8 bytes)
                    MPU6050_Read_Accel();
                    Construct_Flash_Struct(&sCurrentData);
                    g_untDataToWrite = 0;
                    // Copy your struct bytes into the 64-bits variable
                                                                  safely
```

```
                    memcpy(&g_untDataToWrite,&sCurrentData,
                                        sizeof(tsCurrentFlashStruct));
                    Dynamic_Buffer_Write(g_untDataToWrite);
             }

      if(g_untIOButtonPressed)
      {
             g_untIOButtonPressed=0;
             if(!g_int_LogSessionBusyFlag)
             {
                    g_int_SampleCounter=0;
                    g_untCurrentFlashAddress= FLASH_PAGE54_START +
                                  g_int_ButtonPressedCount * PAGE_SIZE;
                    if(g_int_ButtonPressedCount<10)
                    {
                           g_int_ButtonPressedCount++;
                    }
                    g_int_LogSessionBusyFlag=1;
                    g_untState=EN_STATE_PAST_100_SAMPLES;
             }
      }

      switch(g_untState)
      {
      case EN_STATE_PAST_100_SAMPLES:
             Flash_Write_Past_1s();
             g_untState=EN_STATE_NEXT_150_SAMPLES;
             break;

      case EN_STATE_NEXT_150_SAMPLES:
             if(g_int_ReceiveIncomingDataFlag)
             {
                    Flash_Write_After_2Secs(g_untCurrentFlashAddress);
                    g_int_SampleCounter++;
             }
             break;
      }
```

iv. Results

According to requirements, the last 10 pages of the Nucleo-G071RB which are from 54 to 63rd pages are used since the board has 128 KB Flash Memory(See Appendix B). The start addresses of the pages are shown below.

- PAGE 63: 0x0801F800
- PAGE 62: 0x0801F000
- PAGE 61: 0x0801E800
- PAGE 60: 0x0801E000
- PAGE 59: 0x0801D800
- PAGE 58: 0x0801D000
- PAGE 57: 0x0801C800
- PAGE 56: 0x0801C000
- PAGE 55: 0x0801B800
- PAGE 54: 0x0801B000

The verification is done in parts. Firstly, parsing of past 100 data is inspected. When program is started, the program starts parsing from page 54 which is 0x0801B00. In order to verify

whether the first 100 data is parsed into page 54, I checked the address of 100th data is calculated.

$$Past\ 100\ Samples\ Memory\ Size = 8bytes * 100 = (320)_{16}$$
$$The\ 100^{th}\ data's\ address = 0x0801B000 + 320 = 0x0801B320$$

When the user button is pressed, the slot at 0x0801B320 is expected to be filled. After pressing the user button, all newest 100 data is stored in page 54 from 0x0801B000 to 0x0801B320(See Figure 8 and 9). After completing first part of the program, the second part of the program which is parsing next 150 data is executed. The last data's memory address is calculated below, to verify whether all 250 data is stored in page 54.

$$Next\ 150\ Samples\ Memory\ Size = 8bytes * 150 = (4B0)_{16}$$
$$250th\ samples\ Memory\ Place\ in\ Page\ 54 = 0x0801B00 + 320 + 4B0 = 0x08017D0$$

According to Figure 10 and 11, the all the data is stored in page 54 properly. When the user button is pressed again, the 250 data is stored in Page 55(See Figure 12 and 13). According to data structure, the values that represents each fields can be seen.

At the end of the project, I realized that the 2.5 second process was completed in approximately 7 seconds which doesn't satisfy the requirement (See Figure 12 and 13). Locking and Unlocking data or writing data from the start takes many cycles. After realizing that the requirement could be improved by expanding the data reception period. In addition, in second part, all of the 150 data could be parsed in packets of 50 data results in less times of locking or unlocking flash memory.

0x0801B000 : 0x801B000 <Hex> × | New Renderings...

Address	0 - 3	4 - 7	8 - B	C - F
0801AFF0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0801B000	021AA200	0E030500	F1199700	0E030500
0801B010	F6198600	0E030500	EC196500	0E030500
0801B020	011AB000	0E030500	F7197E00	0E030500
0801B030	E4198400	0E030500	E0197600	0E030500
0801B040	DE199D00	0E030500	EF198100	0E030500
0801B050	ED19A400	0E030500	D8199F00	0E030500
0801B060	DD19C200	0E030500	EB199900	0E030500
0801B070	E4197200	0E030500	E6195300	0E030500
0801B080	E3193F00	0E030500	DC196F00	0E030500
0801B090	E619A400	0E030500	DE199600	0E030500
0801B0A0	E8198C00	0E030500	E319AB00	0E030500
0801B0B0	D8197600	0E030500	EC195700	0E030500
0801B0C0	F9198900	0E030500	E7198400	0E030500
0801B0D0	E2197A00	0E030500	F8195800	0E030500
0801B0E0	E4195A00	0E030500	D9194500	0E030500
0801B0F0	D8196F00	0E030500	DA198A00	0E030500
0801B100	D819E500	0E030500	E019B800	0E030500
0801B110	D719A300	0E030500	E1198100	0E030500
0801B120	DF19AE00	0E030500	E019AB00	0E030500
0801B130	F4197E00	0E030500	FE196300	0E030500
0801B140	041A6600	0E030500	EE197300	0E030500
0801B150	F319FA00	0E030500	041AB700	0E030500
0801B160	031A9700	0E030500	DB19BF00	0E030500
0801B170	D919BF00	0E030500	DE19A000	0E030500
0801B180	E019B200	0E030500	0C1AA200	0E030500
0801B190	F619D200	0E030500	ED19C700	0E030500
0801B1A0	F0197000	0E030500	EE198F00	0E030500

Figure 8 First Part of the Page 54

0x0801B000 : 0x801B000 <Hex> × | New Renderings...

Address	0 - 3	4 - 7	8 - B	C - F
0801B1A0	F0197000	0E030500	EE198F00	0E030500
0801B1B0	FF195100	0E030500	FE195B00	0E030500
0801B1C0	031A8900	0E030500	001A9700	0E030500
0801B1D0	F1199600	0E030500	011AA900	0E030500
0801B1E0	FB198900	0E030500	F0198900	0E030500
0801B1F0	FA197E00	0E030500	E519B200	0E030500
0801B200	E5197D00	0E030500	FF197800	0E030500
0801B210	F7199E00	0E030500	F219A800	0E030500
0801B220	DC199900	0E030500	E5199D00	0E030500
0801B230	F9197700	0E030500	FC197E00	0E030500
0801B240	FD198C00	0E030500	F9197B00	0E030500
0801B250	FF197B00	0E030500	F4199A00	0E030500
0801B260	F0198F00	0E030500	FD196D00	0E030500
0801B270	E919A100	0E030500	E919B200	0E030500
0801B280	071AB000	0E030500	EB19B500	0E030500
0801B290	DF198700	0E030500	CC19AD00	0E030500
0801B2A0	DD19B100	0E030500	DD198400	0E030500
0801B2B0	DC19A800	0E030500	CE199F00	0E030500
0801B2C0	E419AA00	0E030500	EF198F00	0E030500
0801B2D0	DA19A300	0E030500	CB198A00	0E030500
0801B2E0	D3196B00	0E030500	E9198500	0E030500
0801B2F0	E719BC00	0E030500	EE19BC00	0E030500
0801B300	041ABB00	0E030500	1D1AAA00	0E030500
0801B310	121AD400	0E030500	091AEC00	0E030500
0801B320	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0801B330	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF

Figure 9 Second Part of the Page 54

0x0801B000 : 0x801B000 <Hex> × | New Renderings...

Address	0 - 3	4 - 7	8 - B	C - F
0801B310	121AD400	0E030500	091AEC00	0E030500
0801B320	B1194B00	0E030500	E119A700	0E043300
0801B330	D0196000	0E050200	EA198C00	0E050600
0801B340	E3196500	0E050F00	FD196300	0E051800
0801B350	D4196100	0E051800	D9196100	0E051800
0801B360	D9193B00	0E051800	C4192200	0E051800
0801B370	D6193400	0E051800	D6190A00	0E051800
0801B380	D5194C00	0E051800	E3191400	0E051800
0801B390	CC195900	0E051800	CD192500	0E051800
0801B3A0	AC195F00	0E051800	E2195300	0E051800
0801B3B0	E0197600	0E051800	D3195D00	0E051800
0801B3C0	D8196800	0E051800	D7196400	0E051800
0801B3D0	D6197C00	0E051800	EB192D00	0E051900
0801B3E0	E2193B00	0E051900	F5195B00	0E051900
0801B3F0	DF193000	0E051900	CC19FDFF	0E051900
0801B400	CB191400	0E051900	DC195000	0E051900
0801B410	E6191B00	0E051900	CE195D00	0E051900
0801B420	E1196800	0E051900	DB192200	0E051900
0801B430	E1194100	0E051900	DA196800	0E051900
0801B440	DF195A00	0E051900	CD196E00	0E051900
0801B450	E2193E00	0E051900	E9193B00	0E051900
0801B460	C7194100	0E051900	E5193400	0E051900
0801B470	DF193700	0E051900	D1196100	0E051900
0801B480	D6195300	0E051A00	DD194100	0E051A00
0801B490	E9195000	0E051A00	E6195700	0E051A00
0801B4A0	D2193A00	0E051A00	DB195600	0E051A00
0801B4B0	F1196800	0E051A00	D0198300	0E051A00
0801B4C0	D2193400	0E051A00	CE193700	0E051A00

Figure 10 Third Part of the Page 54

0x0801B000 : 0x801B000 <Hex> × New Renderings...

Address	0 - 3	4 - 7	8 - B	C - F
0801B5E0	DA191400	0E051C00	E4194C00	0E051C00
0801B5F0	DF192600	0E051C00	ED193E00	0E051C00
0801B600	D4193A00	0E051C00	E0193E00	0E051C00
0801B610	DD195E00	0E051C00	D0191F00	0E051C00
0801B620	D5197500	0E051C00	D1194500	0E051C00
0801B630	DE193400	0E051C00	C7193300	0E051C00
0801B640	DA192D00	0E051C00	D0197800	0E051C00
0801B650	CF194F00	0E051C00	CF191400	0E051C00
0801B660	F1190000	0E051C00	DE193000	0E051C00
0801B670	E0195700	0E051C00	F0193100	0E051D00
0801B680	D9196F00	0E051D00	E5193B00	0E051D00
0801B690	E2195E00	0E051D00	E8191100	0E051D00
0801B6A0	EA193100	0E051D00	CE192600	0E051D00
0801B6B0	E0195000	0E051D00	E0190D00	0E051D00
0801B6C0	E6194200	0E051D00	C3192200	0E051D00
0801B6D0	CF191F00	0E051D00	D9190600	0E051D00
0801B6E0	ED193B00	0E051D00	E7193B00	0E051D00
0801B6F0	D7194200	0E051D00	E0194900	0E051D00
0801B700	CC198A00	0E051D00	D4192600	0E051D00
0801B710	DC196400	0E051D00	E0192600	0E051D00
0801B720	DE192200	0E051D00	DA191B00	0E051E00
0801B730	E9195300	0E051E00	DC193700	0E051E00
0801B740	D7192D00	0E051E00	E4192200	0E051E00
0801B750	D5191100	0E051E00	D3193400	0E051E00
0801B760	CE193000	0E051E00	DA190D00	0E051E00
0801B770	E5198100	0E051E00	D9195300	0E051E00
0801B780	F9192300	0E051E00	C9192900	0E051E00
0801B790	D6197900	0E051E00	E2194900	0E051E00
0801B7A0	D7194100	0E051E00	D4195600	0E051E00
0801B7B0	DA193700	0E051E00	DB194200	0E051E00
0801B7C0	E6195700	0E051E00	E9196800	0E051E00
0801B7D0	E7193800	0E051F00	FFFFFFFF	FFFFFFFF
0801B7E0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0801B7F0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0801B800	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0801B810	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF

Figure 11 Last Part of Page 54

0x0801B800 <Hex> | 0x0801B800 : 0x801B800 <Signed Integer> | New Re

Address	0	2	4	6	8	A	C	E
0801B7C0	6630	87	1294	30	6633	104	1294	30
0801B7D0	6631	56	1294	31	-1	-1	-1	-1
0801B7E0	-1	-1	-1	-1	-1	-1	-1	-1
0801B7F0	-1	-1	-1	-1	-1	-1	-1	-1
0801B800	6648	-84	1550	50	6631	-56	1550	50
0801B810	6629	-66	1550	50	6632	-111	1550	50
0801B820	6646	-130	1550	50	6643	-77	1550	50
0801B830	6640	-73	1550	50	6615	-111	1550	50
0801B840	6616	-125	1550	50	6627	-80	1550	50
0801B850	6635	-76	1550	50	6635	-76	1550	50
0801B860	6645	-52	1550	50	6649	-66	1550	50
0801B870	6642	-126	1550	50	6639	-88	1550	50
0801B880	6637	-35	1550	50	6627	-45	1550	50
0801B890	6647	-109	1550	50	6633	-87	1550	50
0801B8A0	6602	-79	1550	50	6636	-91	1550	50
0801B8B0	6629	-108	1550	50	6619	-83	1550	50
0801B8C0	6639	-77	1550	50	6641	-77	1550	50
0801B8D0	6640	-24	1550	50	6645	7	1550	50
0801B8E0	6629	-125	1550	50	6646	-126	1550	50
0801B8F0	6663	-98	1550	50	6633	-62	1550	50
0801B900	6628	-80	1550	50	6626	-87	1550	50
0801B910	6610	-79	1550	50	6635	-45	1550	50
0801B920	6640	-105	1550	50	6638	-84	1550	50
0801B930	6634	-101	1550	50	6637	-101	1550	50
0801B940	6644	-77	1550	50	6648	-66	1550	50
0801B950	6604	-100	1550	50	6609	-69	1550	50
0801B960	6632	-70	1550	50	6635	-115	1550	50
0801B970	6636	-87	1550	50	6633	-59	1550	50

Figure 12 First Part of Page 55

0x0801B800 <Hex> | 0x0801B800 : 0x801B800 <Signed Integer> | New Rer

Address	0	2	4	6	8	A	C	E
0801BE60	6627	14	1550	55	6613	-34	1550	55
0801BE70	6603	-10	1550	55	6630	-17	1550	55
0801BE80	6638	21	1550	55	6620	69	1550	55
0801BE90	6623	-13	1550	55	6629	24	1550	55
0801BEA0	6632	10	1550	55	6633	-17	1550	55
0801BEB0	6628	20	1550	55	6639	-13	1550	55
0801BEC0	6613	10	1550	55	6619	-13	1550	55
0801BED0	6639	-38	1550	55	6643	31	1550	55
0801BEE0	6605	10	1550	56	6634	-10	1550	56
0801BEF0	6605	-27	1550	56	6629	-31	1550	56
0801BF00	6639	38	1550	56	6624	10	1550	56
0801BF10	6615	-55	1550	56	6603	20	1550	56
0801BF20	6617	83	1550	56	6647	21	1550	56
0801BF30	6623	34	1550	56	6612	-13	1550	56
0801BF40	6635	24	1550	56	6615	76	1550	56
0801BF50	6617	-20	1550	56	6641	52	1550	56
0801BF60	6622	17	1550	56	6618	24	1550	56
0801BF70	6635	24	1550	56	6623	52	1550	56
0801BF80	6618	59	1550	56	6636	10	1550	56
0801BF90	6618	45	1550	57	6626	45	1550	57
0801BFA0	6625	27	1550	57	6634	6	1550	57
0801BFB0	6622	41	1550	57	6620	38	1550	0x801E
0801BFC0	6643	-3	1550	57	6608	13	1550	57
0801BFD0	6630	55	1550	57	-1	-1	-1	-1
0801BFE0	-1	-1	-1	-1	-1	-1	-1	-1
0801BFF0	-1	-1	-1	-1	-1	-1	-1	-1

Figure 13 End of the Page 55

IV. Conclusions

During internship, I benefited from what I learned in EE321, EE497, CS201 and CS443. EE321 helped me to use peripherals such as Interrupts and I2C communication protocol. CS201 helped me to use one of the FIFO methods which is circular buffer. Thanks to circular buffer, I could store the newest 100 data without data loss. EE497 and CS443 gave me the fundamental knowledge for Memory maps and registers.

I improved myself on STM32 technology by expanding my knowledge on reset mechanisms, incident recording and scheduling. I also learned how to use I2C protocol by reading datasheets of GY-521. During this project, I used DO-178C Aviation and MISRA-C (2012 Edition) Standards to learn how to follow safety procedures and to write clean code. These topics will be beneficial for Embedded Software Engineers which I plan to be an expert on.

TEI is a good choice for someone who wants to have a steady work life. There are approximately 30 computer engineers in Software Development Department, that have no need for many software engineers or control engineers. The people in my department understood my passion in embedded software. Consequently, I am happy for completing my internship in TEI.

Appendix

APPENDIX A- MPU-6050 Register Map and Descriptions

InvenSense	MPU-6000/MPU-6050 Register Map and Descriptions	Document Number: RM-MPU-6000A-00 Revision: 4.2 Release Date: 08/19/2013

Addr (Hex)	Addr (Dec.)	Register Name	Serial I/F	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
3B	59	ACCEL_XOUT_H	R	ACCEL_XOUT[15:8]							
3C	60	ACCEL_XOUT_L	R	ACCEL_XOUT[7:0]							
3D	61	ACCEL_YOUT_H	R	ACCEL_YOUT[15:8]							
3E	62	ACCEL_YOUT_L	R	ACCEL_YOUT[7:0]							
3F	63	ACCEL_ZOUT_H	R	ACCEL_ZOUT[15:8]							
40	64	ACCEL_ZOUT_L	R	ACCEL_ZOUT[7:0]							
41	65	TEMP_OUT_H	R	TEMP_OUT[15:8]							
42	66	TEMP_OUT_L	R	TEMP_OUT[7:0]							
43	67	GYRO_XOUT_H	R	GYRO_XOUT[15:8]							
44	68	GYRO_XOUT_L	R	GYRO_XOUT[7:0]							
45	69	GYRO_YOUT_H	R	GYRO_YOUT[15:8]							
46	70	GYRO_YOUT_L	R	GYRO_YOUT[7:0]							
47	71	GYRO_ZOUT_H	R	GYRO_ZOUT[15:8]							
48	72	GYRO_ZOUT_L	R	GYRO_ZOUT[7:0]							
49	73	EXT_SENS_DATA_00	R	EXT_SENS_DATA_00[7:0]							
4A	74	EXT_SENS_DATA_01	R	EXT_SENS_DATA_01[7:0]							
4B	75	EXT_SENS_DATA_02	R	EXT_SENS_DATA_02[7:0]							
4C	76	EXT_SENS_DATA_03	R	EXT_SENS_DATA_03[7:0]							
4D	77	EXT_SENS_DATA_04	R	EXT_SENS_DATA_04[7:0]							
4E	78	EXT_SENS_DATA_05	R	EXT_SENS_DATA_05[7:0]							
4F	79	EXT_SENS_DATA_06	R	EXT_SENS_DATA_06[7:0]							
50	80	EXT_SENS_DATA_07	R	EXT_SENS_DATA_07[7:0]							
51	81	EXT_SENS_DATA_08	R	EXT_SENS_DATA_08[7:0]							
52	82	EXT_SENS_DATA_09	R	EXT_SENS_DATA_09[7:0]							
53	83	EXT_SENS_DATA_10	R	EXT_SENS_DATA_10[7:0]							
54	84	EXT_SENS_DATA_11	R	EXT_SENS_DATA_11[7:0]							
55	85	EXT_SENS_DATA_12	R	EXT_SENS_DATA_12[7:0]							
56	86	EXT_SENS_DATA_13	R	EXT_SENS_DATA_13[7:0]							
57	87	EXT_SENS_DATA_14	R	EXT_SENS_DATA_14[7:0]							
58	88	EXT_SENS_DATA_15	R	EXT_SENS_DATA_15[7:0]							
59	89	EXT_SENS_DATA_16	R	EXT_SENS_DATA_16[7:0]							
5A	90	EXT_SENS_DATA_17	R	EXT_SENS_DATA_17[7:0]							
5B	91	EXT_SENS_DATA_18	R	EXT_SENS_DATA_18[7:0]							
5C	92	EXT_SENS_DATA_19	R	EXT_SENS_DATA_19[7:0]							
5D	93	EXT_SENS_DATA_20	R	EXT_SENS_DATA_20[7:0]							
5E	94	EXT_SENS_DATA_21	R	EXT_SENS_DATA_21[7:0]							
5F	95	EXT_SENS_DATA_22	R	EXT_SENS_DATA_22[7:0]							
60	96	EXT_SENS_DATA_23	R	EXT_SENS_DATA_23[7:0]							
63	99	I2C_SLV0_DO	R/W	I2C_SLV0_DO[7:0]							
64	100	I2C_SLV1_DO	R/W	I2C_SLV1_DO[7:0]							
65	101	I2C_SLV2_DO	R/W	I2C_SLV2_DO[7:0]							
66	102	I2C_SLV3_DO	R/W	I2C_SLV3_DO[7:0]							

Document: MPU-6000-Register-Map1

APPENDIX B- RM0444 Reference manual

Figure 2. Memory map

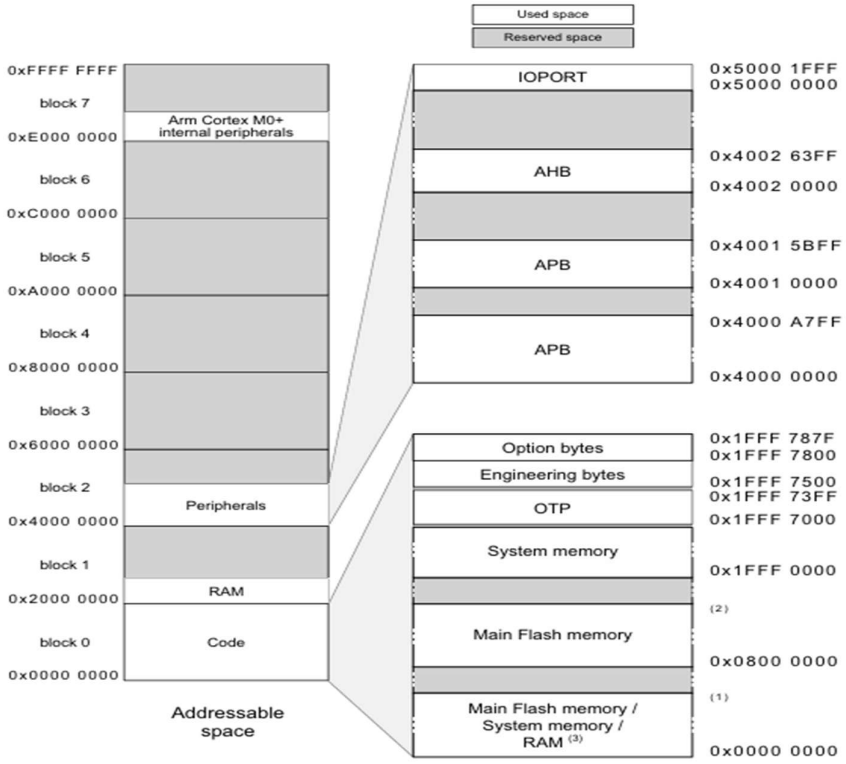


Table 9. Flash memory organization for single-bank devices (continued)

Area	Addresses	Size (bytes)	16 KB devices	32 KB devices	64 KB devices	128 KB devices	256 KB devices (1)
Main memory (Bank 1)	0x0803 F800 - 0x0803 FFFF	2 K					Page 127
	...	...					...
	0x0801 F800 - 0x0801 FFFF	2 K				Page 63	
	...	...				...	
	0x0800 F800 - 0x0800 FFFF	2 K			Page 31		
	...	...			...		
	0x0800 7800 - 0x0800 7FFF	2 K		Page 15			
	...	...		...			
	0x0800 3800 - 0x0800 3FFF	2 K	Page 7				
	...	...	...				
	0x0800 0800 - 0x0800 0FFF	2 K	Page 1				
	0x0800 0000 - 0x0800 07FF	2 K	Page 0				

1. Flash memory configured in single-bank mode, by clearing the DUAL_BANK option bit.

Document: STM32G0x1 advanced Arm®-based 32-bit MCUs STMicroelectronics,2024.

References

STMicroelectronics Community contributors. (2021). Hal Vs LL. In STMicroelectronics Community. STMicroelectronics. Retrieved August 2025, from https://community.st.com/t5/stm32-mcus-products/hal-vs-ll/td-p/432434

Yahboom. (n.d.). Introduction to HAL library and LL library. In Yahboom. Yahboom. Retrieved August 2025, from http://www.yahboom.net/public/upload/upload-html/1701776660/Introduction%20to%20HAL%20library%20and%20LL%20library.html

STMicroelectronics. (n.d.). STM32CubeIDE. In STMicroelectronics. STMicroelectronics. Retrieved August 2025, from https://www.st.com/en/development-tools/stm32cubeide.html

TEI. (n.d.). Hakkımızda. In TEI. TEI. Retrieved August 2025, from https://www.tei.com.tr/kurumsal/hakkimizda

TEI. (n.d.). Products. In TEI. TUSAŞ Engine Industries, Inc. Retrieved August 2025, from https://www.tei.com.tr/en/products